
Architecting with AI
From Requirements to Reality

> *I asked AI to write my conference bio. It called me a visionary thought leader who disrupts paradigms at scale. So... I left it.*

Because honestly? Who's going to argue with that?

⚠ A Word of Caution

AI is confident. AI is fast.

AI will absolutely write something that sounds incredible

and is **completely disconnected from reality.**

Key Risk

That's true for bios. **And it's very, very true for architecture.**

THESIS STATEMENT

> **The best architects I know use AI not to replace their thinking, but to accelerate their judgment.**

*Okay. Now let's **actually** talk about how to do that.*



Chad Green

Senior Systems Architect
Jasper Engines & Transmissions

Meet the Speaker

35+
Years building production systems

~3
Years serious AI integration

About Chad

- Software architect with 35+ years of building systems that had to **actually work** in production
- Passionate about making **complex architecture** accessible and **fun**
- Early adopter of AI-assisted development; I have seen **the good, the bad, and the hallucinated**
- Conference speaker, community builder, **occasional over-prompter**
- **Superpower**: Turning **“we need a system that does everything”** into something buildable

Session Overview

Agenda

Five sections — from toolkit to multi-agent workflows — all grounded in a single demo feature

Prologue

The Architect's AI Toolkit

Right tool, right job, every time

Chapter One

Requirements Are Lying to You

Clarifying requirements, surfacing hidden constraints

Chapter Two

Design Options, Tradeoffs, and Adversarial Insight

Exploring architecture options, stress-testing designs

Chapter Three

Documentation, Governance, and Human Judgment

Full APS walkthrough, AI-assisted documentation, multi-tool approach

Chapter Four

Copilot Squad

Multi-agent architectural workflows

The Demo Feature: Notification Service

Our through-line for every demo today

The User Story

> *As a system user, I want to receive timely notifications through my preferred channel when key events occur, so that I can stay informed and respond promptly.*

Notification Service · Core User Story

Why This Feature

5 Reasons This Works as a Demo

- Every architect has built one
- Smaller user story, rich architecture decisions
- Clean, teachable footprint
- Ai generates it fast, without wall-clock time risk
- Containers real ADR decisions with genuine tradeoffs

What We'll Cover

Requirements → Design → Infrastructure → Documentation → Squad


The Architect's AI Toolkit
Right tool. Right job. Every time.

Pick Your Tool by What It's Grounded In

Every AI tool is optimized for a different kind of input. That determines what it's good at.

GitHub Copilot

Code & Files

What It Knows

Your repository, your templates, your IaC

Best For

Structured docs, code-adjacent tasks, AI-enhanced process workflows

Claude Code/Cowork

Local Files & Long Context

What It Knows

Everything you hand it — up to 1M tokens

Best For

Multi-document generation, cross-file reasoning, dev labs

M365 Copilot

Org Knowledge + Live Web

What It Knows

The internet today + your org's email, Teams, SharePoint

Best For

Research, exec-layer docs, org-specific writing

Other Tools

OpenAI Codex

Google Gemini

Cursor

GitHub Copilot

GitHub Copilot

The architect's IDE-native workhorse

Grounded In: Code, files, repo context, template instruction files

Multi-Model Support


Claude


OpenAI


Gemini


Grok


MAI


Kimi

Model	Capability Tier	Cost Tier
K3	Powerful	Medium
Terra	Versatile	Medium
Sol	Powerful	Medium
Astra	Powerful	Very High

Best For

- AI-Enhanced Workflows
- Resource specifications
- Template-bound generation
- IaC scaffolding

Later in the Presentation

Copilot Squad — multi-agent architectural workflows

Everything in the Architecture Process demo runs on GitHub Copilot.

Claude

Claude & Claude Cowork

Deep context, long-horizon reasoning, local file workflows

Grounded In

Whatever you hand it — local files, pasted documents, cross-app context

Up to 1 million tokens

The Claude Model Lineup

Claude Haiku
Fast, cheap: repetitive tasks, quick drafts, simple analysis

Claude Sonnet
Balanced capability and cost: production-grade architecture reasoning

Claude Opus
Deeper reasoning: adversarial review, complex tradeoff analysis

Claude Fable
God mode: Security reviews, many-step processing

Claude Cowork — Desktop Agent

- **Desktop agent** — Mac and Windows
- **Multi-step local file workflows** — operate across files in one session
- **Hand it 7 template files → receive 7 documents.** One session, one prompt.

Best For

Multi-document generation | **Cross-file reasoning** | **Long context** | **Developer labs** | **Adversarial review**

Chad's Use Case

Multi-document generation in a single task; building developer labs and hands-on exercises.

For the Design Options section, Claude Sonnet and Opus play specific adversarial roles.

M365 Copilot

M365 Copilot

The intelligence layer for your organization and the live web

The Microsoft IQ Stack — Two Grounding Sources

Web IQ

Live web intelligence

- Live web intelligence — search, news, pages, citations
- Built on Bing, re-architected for agents
- Keeps responses current and externally grounded

Work IQ

Your organization's knowledge

- Email, Teams messages, SharePoint documents, meeting notes
- Knows your people, your history, your terminology
- Org-native voice baked into every response

E7 / Frontier Suite

The Combination

Research with live external context, written with your organization's voice baked in.

Chad's Use Cases

- Executive summaries
- Company-specific architecture documents
- Stakeholder research
- Competitive landscape analysis

Why not GitHub Copilot? Copilot doesn't know your org. M365 Copilot does.

When I Reach for Which Tool

The practitioner's cheat sheet

Task	Tool	Model	Why
Requirements analysis	**GitHub Copilot**	**MAI-Code-1**	*File-aware, template-structured*
Architecture exploration	**GitHub Copilot**	**MAI-Code-1**	*Broad, fast, IDE-native*
Second perspective on options	**Claude**	**Sonnet**	*Long-context cross-comparison*
Adversarial review	**Claude**	**Opus**	*Deepest critical reasoning*
APS documentation (Phases 1–3)	**GitHub Copilot**	**MAI-Code-1-Flash**	*Template-bound, structured, fast*
Executive summary	**M365 Copilot**	**Web IQ + Work IQ**	*Org-grounded, stakeholder-aware*
Multi-document batch generation	**Claude Cowork**	**Sonnet**	*1M context, local file access*
Complex architectural reasoning	**Claude or MAI-Thinking-1**	**Opus or Thinking-1**	*Deepest available reasoning*

This is the matrix. Everything you'll see today maps to a row.

What You'll See Today

11 demos across 4 sections

Requirements Are Lying to You	**Demo 1** Requirements Analysis	**Demo 1B** The Correction Moment		
Design Options, Tradeoffs & Adversarial	**Demo 2** Architecture Options	**Demo 2B** Second Perspective	**Demo 3** Adversarial Review	
Documentation, Governance & Human Judgment	**Demo 4** Phase 2 Design	**Demo 5** Phase 3 Infrastructure	**Demo 6** Executive Summary	**Demo 7** ARB Check
Copilot Squad	**Demo 8** Copilot Squad End-to-End			

One feature. Four tools. Nine demos. Ninety minutes.


Requirements Are Lying
Hidden Assumptions in Plain Sight

Requirements Hide Landmines

Ambiguity

Requirements that mean different things to different people: creating silent misalignment before a single line of code is written.

Contradictions

Conflicting rules nobody noticed during review: invisible in isolation, catastrophic when two parts of the system finally meet.

Missing NFRs

The performance, security, and reliability expectations are never written down, but everyone expects them to be met at launch.

Hidden Constraints

Business rules and platform limits that only appear after key decisions are made are too late to allow a cost-effective course correction.

Why Humans Miss Things

Cognitive Load

Juggling business rules, edge cases, constraints, dependencies, timelines, and capacity (simultaneously) is impossible.

Anchoring Bias

We latch onto the first interpretation that makes sense and stop exploring alternatives.

Incomplete Mental Models

We can't ask about constraints we don't realize exist.

AI doesn't get tired, biased, or anchored.

AI as a Requirements Clarifier

Raw Requirements → AI Analysis → Structured Insights

Extracts Ambiguities

Surfaces the hidden meanings in vague language before they become costly assumptions.

Surfaces Hidden NFRs

Infers the performance and security requirements nobody wrote down, but everyone expects.

Identifies Contradictions

Flags rules that conflict with each other; the ones that survive every review until they break production.

Asks Clarifying Questions

The ones we forget to ask, precisely when it matters most, before design decisions are locked in.

AI expands awareness, not assumptions.

Anatomy of a Requirements Prompt

Structure your prompt and you structure your output.

requirements-analysis-prompt.md

```
Analyze the following user story
and acceptance criteria.
Identify:
    Ambiguities
    Contradictions
    Missing acceptance criteria
    Implicit non-functional requirements
    Hidden constraints
    Assumptions the stories made
    Questions an architect should ask
    before designing.
DO invent new features. Only
NOT analyze
what is present or implied.
Ask clarifying questions as you work.
```

Element 1

What to do

Opens with a clear task directive — "Analyze the following..." — so the model knows its role before reading any content.

Element 2

Output structure

The seven-item list shapes the response format — ambiguities, contradictions, NFRs, constraints, assumptions, and clarifying questions.

Element 3

The guardrail

"DO NOT invent new features." Explicit constraint prevents hallucinated requirements from contaminating the analysis.

Element 4

The behavior directive

"Ask clarifying questions as you work." Keeps the model in analyst mode — surfacing gaps rather than silently assuming.

Four elements: *Task. Structure. Guardrail. Behavior. Every good requirements prompt has all four.*

Demo Setup: Requirements Clarification

GitHub Copilot · MAI-Code-1

Feature Notification Service

User Story

As a system user, I want to receive timely notifications through my preferred channel when key events occur, so that I can stay informed and respond promptly."

Tool

GitHub Copilot

What We'll Use

Our structured requirements prompt from the previous slide

The Structured Prompt

requirements-analysis.prompt

```
Analyze the following user story and acceptance criteria.
Identify:
Ambiguities
Contradictions
Missing acceptance criteria
Implicit non-functional requirements
Hidden constraints
Assumptions the stories made
Questions an architect should ask
before designing.
DO NOT invent new features. Only analyze what is
present or implied.
Ask clarifying questions as you work.
```

Task Defines what the AI should do — analyze the user story and AC

Structure Seven output categories create structured, scannable insight

Guardrail "DO NOT invent new features" — prevents hallucinated requirements

Behavior "Ask clarifying questions as you work" — keeps AI probing, not concluding

DEMO 1 NEXT

Demo Setup: Requirements Clarification

GitHub Copilot · MAI-Code-1

What You'll See

- AI analyzing the Notification Service user story
- AI surfacing ambiguities & contradictions
- AI identifying missing NFRs
- AI generating clarifying questions

The Prompt

prompt.txt

```
Analyze the following user story and acceptance criteria.
Identify :
    Ambiguities
    Contradictions
    Missing acceptance criteria
    Implicit non-functional requirements
    Hidden constraints
    Assumptions the stories made
    Questions an architect should ask before
        designing a solution.

DO NOT invent new features. Only analyze what is
present or implied.
Ask clarifying questions as you work.
```

GitHub Copilot

Live Demo

Demo 1

LIVE DEMO 1

Requirements Analysis — Notification Service

Tool
GitHub Copilot

Feature
Notification Service

Task
Requirements Clarification

What AI Got Right / What AI Got Wrong

What It Got Right

- ✓ **Surfaced delivery channel ambiguity** — "preferred channel" is undefined
- ✓ **Identified missing NFRs** — delivery SLA, retry behavior, dead-letter handling
- ✓ **Flagged the opt-out management gap**
- ✓ **Asked about notification ordering and idempotency**

What It Might Get Wrong

- × **Hallucinated constraints** that don't exist
- × **Made platform assumptions** not specified in requirements
- × **Confidently stated architectural conclusions** from a single user story
- × **Over-applied patterns** from unrelated notification systems

AI expands the problem space — and the **nonsense space** *.*
Our job is to separate the two.

The Correction Workflow

When AI gets something wrong — fix it in the conversation, not outside it.

1

Step 1

Identify

Spot the hallucination or incorrect assumption. Name it specifically.

2

Step 2

Isolate

Tell the AI exactly which part is wrong. Don't rewrite the whole prompt.

3

Step 3

Correct with Context

Provide the actual constraint or fact. Let the AI re-reason.

4

Step 4

Re-Validate

Ask AI to confirm the corrected section is consistent with everything else it produced.

Example Correction Prompt

```
"You stated [X]. That constraint doesn't exist in the requirements. Remove it and re-examine only that section in context of the rest of your analysis."
```

Correction in context is faster than restarting. And the conversation history is an asset.

GitHub Copilot

Live Demo

LIVE DEMO 1B

The Correction Moment — Fixing a Hallucination In Context

Tool	Feature	Focus
GitHub Copilot	Notification Service	Hallucination Correction In Context

→ Continuing from Demo 1 — same conversation

Key Takeaway

> **AI expands the problem space — but architects decide what's real.**

AI helps you see more. AI helps you ask better questions. The judgment is still yours.


Design Options, Tradeoffs, and Adversarial Insight
Exploring Strengths, Weaknesses, and Assumptions

AI Widens the Solution Space

AI doesn't anchor. That's the point.

- Generates four or five viable architectures in seconds
- Surfaces patterns you might not have considered
- Compares tradeoffs across quality attributes
- Does all of this without anchoring on the first idea that fits

Demo Setup: Architecture Options

GitHub Copilot · MAI-Code-1

What You'll See

- **AI proposing 4 distinct architecture approaches** for the Notification Service
- AI comparing **strengths and weaknesses** per approach
- AI identifying **key risks** per approach
- AI surfacing **hidden assumptions** per approach

4 approaches evaluated: Modular Monolith · Microservices · Event-Driven · Serverless

The Prompt

GitHub Copilot — MAI-Code-1

```
Given the clarified Notification Service requirements,
propose 4 distinct architecture approaches :
   Modular monolith
   Microservices
   Event-driven
   Serverless
For each , include:
   High-level description
   Strengths
   Weaknesses
   Key risks
   Team skill assumptions
```

Live Demo

2

Architecture Exploration

Architecture Options + Tradeoffs

Notification Service

GitHub Copilot | 4 Architecture Approaches

Demo Setup: Second Perspective

Claude Sonnet

Context: Same clarified Notification Service requirements. Same four architectures requested.

What to Watch For

- Does Claude surface a **fifth approach** Copilot missed?
- Does it **weight the risks differently** ?
- Does it **question assumptions** Copilot accepted?
- Where do they **agree** ? Where do they **diverge** ?

The Prompt

claude-second-perspective.md

```
Given the following clarified requirements for a
Notification Service, propose   4distinct architecture
approaches and evaluate each across:
   Strengths and weaknesses
   Key operational risks
   Team skill requirements
   Hidden assumptions

Do not assume  any existing tech stack.
Challenge all assumptions   present in the requirements.
```

Same Task

Notice the slight prompt variation — *"Do not assume any existing tech stack"* and *"Challenge all assumptions"* direct Claude toward a wider, more adversarial evaluation posture.

Key Prompt Differences vs. GitHub Copilot

"Do not assume any existing tech stack" removes implementation anchoring. **"Challenge all assumptions"** unlocks adversarial evaluation posture — wider options, harder questions.

Claude Sonnet

Live Demo

LIVE DEMO 2B

Second Perspective

Notification Service — Broader Architectural Pattern Knowledge

2B

Feature: **Notification Service**

Same requirements. **Different lens.**

4 architectures requested

The Power of Adversarial AI

After choosing a preferred architecture — stress-test it.

1

Different Model

Use a separate AI perspective to avoid confirmation bias.

2

Different Perspective

Explicitly instruct it to attack, not validate, your design.

3

Brutally Honest

It will surface risks, gaps, and failure modes without sugarcoating.

4

Finds Blind Spots

Hidden assumptions, scaling risks, security gaps, data consistency issues.

The closest thing to a pre-mortem without deploying to production.

Demo Setup: Adversarial Review

Claude Opus

What You'll See

- **AI attacking** the chosen Notification Service architecture
- **Hidden assumptions** surfaced and challenged
- **Scaling risks** identified with specific failure scenarios
- **Security gaps and data consistency issues** named explicitly
- **Operational failure modes** described in production terms

⚠ Every finding rated: **Critical** / **High** / Medium — no sugarcoating.

The Prompt

```
You are a senior architect conducting an adversarial
review of the following architecture proposal.
Your job is NOT to validate this design.
Your job is to find every flaw, gap, hidden
assumption, and failure mode you can identify.
For each issue found:
Name the specific risk
Explain the failure scenario
Identify who or what is harmed
Rate severity: Critical / High / Medium
Be specific. Be brutal. Do not sugarcoat.
If this architecture went to production as described,
what would break and when?
```

The closest thing to a pre-mortem without deploying to production.

Claude Opus

Live Demo 3

3

Adversarial Review

Notification Service — Stress-Testing the Architecture

Find Every Flaw

Severity-Rated Findings

Failure Mode Analysis

Adversarial Review Debrief

What Claude Opus found in the Notification Service architecture

Critical
2 findings

- **No delivery guarantee defined**
  "Reliable" is not a delivery contract
- **Retry storm risk**
  Unbounded retry under load will cascade to downstream services

High
3 findings

- **Opt-out management absent**
  GDPR/CAN-SPAM compliance gap
- **Channel fallback strategy undefined**
  No fallback when primary channel is unavailable
- **Dead-letter queue ownership unassigned**
  Failed notifications disappear silently

Medium
3 findings

- **Notification ordering not guaranteed**
  No ordering contract defined for event consumers
- **Idempotency not addressed**
  Duplicate delivery possible on retry
- **Observability scope unclear**
  Delivery audit trail not specified

Every Critical and High item above is a real ARB rejection point. *We caught them in Demo 3, not at the board.*

When to Use Which for Architecture

A practitioner's guide for the exploration and review phases.

Exploration Phase | Review Phase | Documentation Phase

Task	Why
Initial architecture options	Grounded in your repo and existing tech stack
Broadening the option set	Wider architectural pattern knowledge, 1M context
Adversarial / stress-test review	Deepest critical reasoning, no diplomatic softening
Tradeoff documentation (ADR)	Template-bound, structured, IDE-native

The same task through three different tools gives you breadth, depth, and documentation. Use all three.

Key Takeaway

> **AI challenges us in ways humans rarely do — and that makes our designs stronger.**

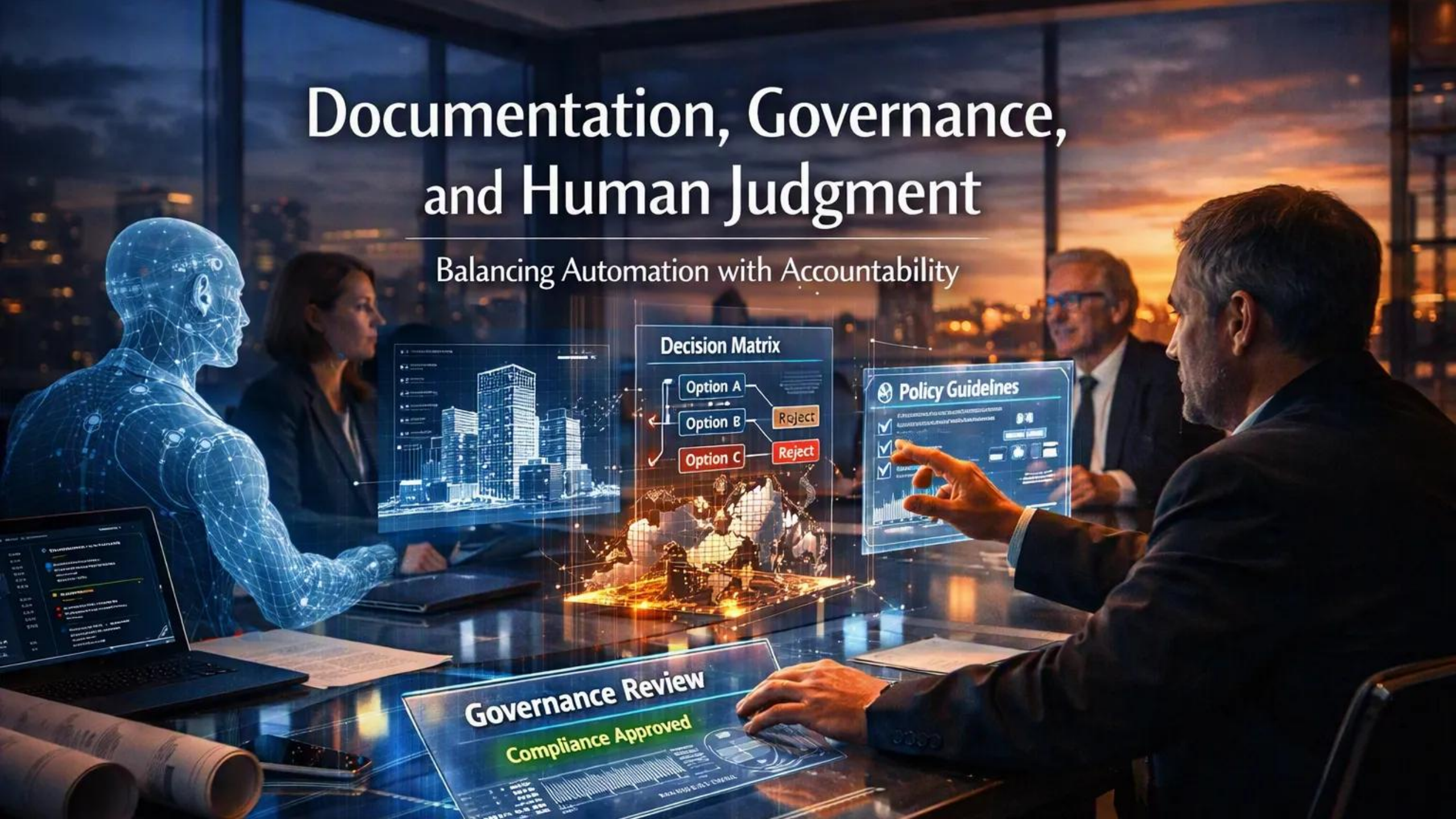
AI gives you breadth. Adversarial AI gives you depth.

Together they build more resilient, defensible architectures.

Breadth — AI Exploration | **Depth — Adversarial Review** | **Outcome — Resilient Architecture**

Documentation, Governance, and Human Judgment

Balancing Automation with Accountability



CAUTION — The Danger of AI Slop

Panel 01

Volume Without Clarity

AI can generate too much — volume without clarity is noise, not documentation.

Panel 02

Teams Drown in Docs

When everything is documented, nothing is documented.

Panel 03

Architects Must Curate

We decide what matters, what belongs, and what gets cut.

AI is a drafting engine — not a decision-maker.

Templates + Instructions + Examples = Quality

Structure → Clarity → Quality = AI Slop Eliminated

Prevent AI Slop

Constrain → Focus

How: Templates

A template tells AI exactly **what to write** — and critically, **what not to write** . Constrained output replaces unconstrained hallucination.

Enforce Structure

Ground → Govern

How: Instructions

Instructions encode **governance rules** AI would otherwise guess at. Every standard, every convention — encoded once, applied every time.

Accelerate Drafting

Speed → Quality

How: Examples

Examples give AI a **calibration target** — not a blank canvas to invent from. Aim at a reference, not at an abstraction.

"AI wrote this" *vs.* **"AI helped me write this"** *— the difference is structure.*

The Jasper Architecture Process

Phase 0
Intake

Architect decides whether new architecture is required

- Intake Assessment
- Go / No-Go Decision

Phase 1
Foundation

Establish the system narrative: guide, scope, context, risks, assumptions

- Architecture Guide
- System Context (C4)
- Risk Register
- Assumption Register
- Non-Goals & Guardrails
- Architecture README

Phase 2
Design
Gate

Detail the components and decisions: specs, journeys, ADRs, contracts

- Architecture Specs
- User Journeys
- Process Flows
- ADR Register
- Event Contracts
- API Contracts

Phase 3
Infrastructure
Gate

Design the Azure footprint: guide, resource specs, capacity, exec summary

- Infrastructure Guide
- Resource Specifications
- Capacity Planning
- Executive Summary
- Glossary

Phase 4
ARB Gate
Gate

Formal review by the Architecture Review Board. Approve, merge, or send back.

- PR: All Phase 1–3 Docs
- ARB Review (8 criteria)
- Approve / Reject

Phase 5
Handoff

Parallel handoffs to Tech Lead, DevOps, and Product Owner

- Tech Lead Brief
- DevOps Package
- Product Owner Summary

AI fits into each phase — but it doesn't replace any phase.

The Jasper Architecture Process

Click 2: Governance Gates

Phase 0 — Intake

- Architect decides whether new architecture is required
- Pass / No-pass decision
- Proceeds to Phase 1 on pass

Phase 1 — Foundation

- Architecture Guide
- System Context (C4)
- Risk Register (init)
- Assumption Register (init)
- Architecture README

Phase 2 — Design

Gate

- **Architecture Specs + ADRs**
- **User Journeys + Process Flows**
- **Event + API Contracts**
- **All ADRs at "proposed"**

Phase 3 — Infrastructure

Gate

- **Infrastructure Guide (RTO/RPO)**
- **Resource Specifications**
- **Capacity + Cost Estimates**
- **Executive Summary**

Phase 4 — ARB Gate

Formal Review

- **Architecture Review Board**
- **Approve · Merge · Send back**
- **ADRs flip to "accepted"**
- **8-criteria completeness check**

Phase 5 — Handoff

- Technical Lead handoff
- DevOps handoff
- Product Owner handoff
- Parallel execution

Gate = Formal Checkpoint | Phases 2, 3, and 4 are governance gates — not formalities. All documents must be complete before advancing. | **3 Gates · 3 Quality Checkpoints**

AI fits into each phase — but it doesn't replace any phase.

The Jasper Architecture Process

Review Cadence + AI Role — who drafts and who decides at every phase



AI drafts — architects and the ARB decide — the cadence makes that division of labor explicit.

Phase 0: Intake

PHASE 0
Intake
Architect decides whether new architecture is required

PHASE 1
Foundation
System narrative, guide, scope, context, risks, assumptions

PHASE 2
Design
Components, decisions, specs, journeys, ADRs, contracts
Gate

PHASE 3
Infrastructure
Azure footprint, resource specs, capacity, exec summary
Gate

PHASE 4
ARB Gate
Formal Architecture Review Board — approve, merge, or send back
Gate

PHASE 5
Handoff
Parallel handoffs to Tech Lead, DevOps, and Product Owner

Intake Decision — Notification Service

The architect evaluates the Notification Service request: is new architecture genuinely required, or can it be served by an existing system? This is not an AI task — it is a human judgment call. If the answer is yes, we proceed.

Result: New architecture required — Notification Service proceeds to Phase 1.

The Feature: Notification Service

Everything from here runs live on this feature.

User Story

"As a system user, I want to receive timely notifications through my preferred channel when key events occur, so that I can stay informed and respond promptly."

What We Know From Prior Sections

- **Delivery channel is undefined** — a real architectural decision
- **Delivery guarantee semantics are missing** — at-least-once? exactly-once?
- **Opt-out management is absent** — regulatory exposure
- **Event-driven architecture selected** as the preferred approach
- CRITICAL Retry storm · Dead-letter ownership gap
  HIGH Idempotency · Ordering

What We're About to Do

Phase 1 **Build the foundation**

Demo 4 — Architecture Guide, System Context, Risk & Assumption Registers

Phase 2 **Design the decisions**

Demo 5 — ADRs, User Journeys, Event Contracts

Phase 3 **Design the infrastructure**

Demo 6 — Resource Specs + Demo 7 — Executive Summary (M365 Copilot)

Phase 4 **Run the ARB completeness check**

Demo 8 — Full package reviewed against 8 ARB acceptance criteria

Requirements analysis complete. Design selected. Now we document — with AI doing the drafting and architects doing the authoring.

Phase 1: Foundation

Phase 0 Intake | **Phase 1 Foundation** | Phase 2 ⊕ Design | Phase 3 ⊕ Infrastructure | Phase 4 ⊕ ARB Gate | Phase 5 Handoff

Phase 1 Document Sequence — 7 Documents

1. **Architecture Guide**
2. **Non-Goals and Guardrails**
3. **System Context and Logical Components (C4)**
4. **Risk Register** (initialized)
5. **Assumption Register** (initialized)
6. **Architecture README**
7. **Docs README**

AI Role in Phase 1

- Drafts each document using template + instructions
- Proposes risks and assumptions for review
- Generates C4 diagram context from requirements
- Architect reviews, corrects, and approves each output

Exit Gate

All foundation documents complete and approved before Phase 2 begins.

Demo Setup: Phase 1 Foundation

GitHub Copilot · MAI-Code-1-Flash

What You'll See

- **Architecture Guide** generated from template + instructions
- **System Context (C4 Level 1)** generated from requirements
- **Risk Register** seeded by AI with Notification Service-specific risks
- **Assumption Register** seeded by AI with implementation assumptions
- **Architect review pass** — what gets edited, what gets kept

Pattern

Each Phase 1 document uses the same pattern — template reference, instructions file, example, context injection.

The Prompt

copilot-prompt.md

```
Using the Architecture Guide template, instructions,
and example (docs-templates/architecture-guide) ,
generate the Architecture Guide for the
Notification Service .

// Context injection
Requirements context:
  [clarified requirements] .

Architecture decision:
  event-driven approach selected.

Known risks:
  [from adversarial review output] .
```

Template + instructions + context injection — the same pattern for every Phase 1 document.

GitHub Copilot

Live Demo

DEMO 4

Phase 1 Foundation

Notification Service

Architecture Guide · System Context (C4) · Risk Register (seed) · Assumption Register (seed)

Tool: GitHub Copilot · MAI-Code-1-Flash

What AI Seeded vs. What You Corrected

Phase 1 — Foundation

The architect's edit pass is not optional — it's the job.

What AI Got Right

- ✓ **Architecture Guide narrative:** Accurate, on-structure, tone appropriate
- ✓ **C4 system boundary placement:** Notification Service correctly scoped
- ✓ **Risk Register seeds:** Retry storm, dead-letter ownership, GDPR gap — all correctly proposed
- ✓ **Assumption Register:** Team familiarity with Service Bus correctly flagged

What Needed a Human

- **Guiding principles:** AI-generated generic principles; replaced with Jasper-specific ones
- **C4 downstream channel adapters:** AI omitted the SMS gateway; added manually
- **One assumption was actually a decision:** Already made — moved to ADR register
- **Risk severity ratings:** AI rated two risks as Medium — clearly High given org context

AI drafted the document. You authored it. The distinction matters.

GitHub Copilot

Phase 2: Design

Full decision record: specs, journeys, ADRs, and event contracts — before advancing to infrastructure



Phase 2 Documents

Core Documents

- Architecture Specifications
- User Journeys — one per primary persona per major feature, with NFR callouts
- Process Flows
- ADR Register
- API Contracts

ADRs — Notification Service

- Broker selection
- Delivery guarantee model
- Channel adapter pattern
- Retry strategy
- Opt-out management approach

Event Contracts

- Notification Requested
- Notification Delivered
- Notification Failed

Exit Gate

All design documents complete · All ADRs at **"proposed"** status before advancing to Phase 3

Phase 2 produces the full decision record before infrastructure begins.

Demo Setup: Phase 2 Design

GitHub Copilot · MAI-Code-1-Flash

GitHub Copilot

What We'll Generate

ADR: Message Broker Selection
Service Bus vs. Event Hubs vs. Event Grid

User Journey: Notification Receipt
Primary persona — end-to-end flow with NFR callouts

Event Contract: NotificationRequested
Schema, payload, and trigger conditions

Event Contract: NotificationDelivered
Confirmation event with delivery metadata

Prompt Pattern

GitHub Copilot · MAI-Code-1-Flash

```
Using the ADR template, instructions, and example
(docs-templates/adr) , generate an ADR for
the decision to use Azure Service Bus as the
message broker for the Notification Service.

Decision drivers:
   // at-least-once delivery guarantee required
   // dead-letter queue support required
   // message ordering not guaranteed (acceptable)
   // team has existing Service Bus expertise

Options to evaluate:
   - Azure Service Bus
   - Azure Event Hubs
   - Azure Event Grid

// Template: docs-templates/adr
// Include: strengths, weaknesses, consequences,
//          status: proposed, superseded ADRs: none
```

Pattern Note

Decision drivers and options are human-supplied. The structure, tradeoff analysis, and consequence documentation are AI-drafted.

Human-supplied: *decision drivers and options.* **AI-drafted:** *structure, tradeoff analysis, and consequence documentation.*

GitHub Copilot

Live Demo 05

LIVE DEMO 5

Phase 2 Design — ADR, User Journey, Event Contracts

ADR (Broker Selection) · User Journey (Notification Receipt) · Event Contracts

GitHub Copilot · MAI-Code-1-Flash

The Completeness Check

Before advancing to the next phase — let AI verify what's missing.

- Validates all required documents for this phase are present
- Verifies cross-references between documents are consistent
- Flags placeholder values ("TBD," "to be determined," "N/A")
- Confirms every ADR is in the correct status
- Identifies risks or assumptions referenced but not registered

Prompt Pattern

```
// Phase 2 Completeness Check
Review the Phase 2 documents produced for the
Notification Service against the APS Phase 2
exit criteria. Identify:
 Missing required documents
 Broken or missing cross-references
Any remaining TBD or placeholder values
ADRs not at "proposed" status
Risks or assumptions referenced but
not registered
Be specific. List each gap with the
document and section it appears in.
```

A five-minute completeness check prevents a 45-minute ARB rejection conversation.

Phase 3: Infrastructure

Phase 3 Documents

- Infrastructure Guide
  - Regional deployment · HA/DR (real RTO/RPO) · Security · Monitoring · Cost
- Resource Specifications
  - Service Bus · Function App · ACS · Cosmos DB · Key Vault · App Config
- Infrastructure README
- VNet Capacity Planning
- Glossary
- Executive Summary
  - Generated in M365 Copilot
- Repository Root README

AI Tools in Phase 3

GitHub Copilot

Technical documents & resource specs

- Infrastructure Guide
- All six resource specifications
- VNet capacity planning
- README and Glossary

M365 Copilot

Web IQ + Work IQ

- Executive Summary — org-grounded, stakeholder-aware

Exit Gate (Strict)

Real Numbers Required

- RTO and RPO — concrete values, not ranges
- Cost estimates — per resource, per environment
- Capacity numbers — throughput, scale units, tier
- HA/DR strategy fully specified

NO TBDs PASS THIS GATE

Phase 3 requires real numbers — RTO, RPO, cost estimates, capacity. No TBDs pass the exit gate.

GitHub Copilot · M365 Copilot

Demo Setup: Phase 3 — Two Tools, Two Jobs

GitHub Copilot | M365 Copilot

Track A — Technical Documents

GitHub Copilot · MAI-Code-1-Flash

What We'll Generate

- Infrastructure Guide (HA/DR)
- Azure Service Bus Namespace
- Azure Function App
- Azure Communication Services
- Azure Cosmos DB
- Azure Key Vault
- Azure App Configuration

```
// Resource Specification Prompt Pattern Using the Service Bus
Namespace resource specification
template and instructions (docs-templates/infra/service-bus-
namespace) ,
generate the resource specification for the
Notification Service Service Bus namespace.

Environment targets: dev , staging , production .
Tier: Premium .
Naming convention: [org standard] .
Tagging standard: [org standard] .
```

Track B — Executive Layer

M365 Copilot (Web IQ + Work IQ)

What We'll Generate

- Executive Summary — stakeholder-ready, org-grounded narrative

Why M365 Copilot Here

- Work IQ knows your org's language, stakeholders, and existing platform context
- Web IQ grounds current market and technology framing
- GitHub Copilot doesn't know who your executives are or how your company talks about itself

> **The Combination**
>
> Research with live external context, written with your organization's voice baked in. That's what the executive audience needs — and what only M365 Copilot delivers.

Key Principle

- Same phase. Different audiences. Different tools. Both right.

Same phase. Different audiences. Different tools. Both right.

GitHub Copilot

Live Demo

6

Phase 3 — Resource Specifications — Notification Service

Service Bus Namespace | Function App | Azure Communication Services | Cosmos DB | Key Vault | App Configuration

Switching to IDE...

Live

M365 Copilot · Web IQ + Work IQ

LIVE DEMO 7

Phase 3 — Executive Summary — Notification Service

Phase **Phase 3: Infrastructure**

Feature **Notification Service**

Web IQ Grounding

Current market context, technology landscape, industry relevance

Work IQ Grounding

Org terminology, existing platform context, stakeholder framing

Phase 4: ARB Gate — Gate-Ready in Minutes

GitHub Copilot

4 Architecture Review Board Gate · DEMO 8

Phase 4 Overview

PR contains all Phase 1–3 documents

- PR description: feature, architectural approach, infrastructure footprint, key ADRs, risk & assumption counts, completeness checklist

ARB reviews against 8 criteria:

Completeness | Architectural Soundness | Decision Quality | Infrastructure Fitness | Risk Coverage | Standards Compliance | No Duplication | Progressive Disclosure

- **If accepted:** PR merges; ADR statuses flip from "proposed" to "accepted"
- **If rejected:** ARB provides specific, actionable feedback

DEMO 8 **ARB Completeness Check & PR Description**

GitHub Copilot · MAI-Code-1

What we'll generate

- PR description
- ARB completeness check against all 8 acceptance criteria

```
// ARB Completeness Check Prompt
Review the Notification Service architecture package
against the APS Phase 4 ARB acceptance criteria.
For each of the 8 criteria, state:
Pass / Fail / Partial
specific evidence for the rating
if Fail or Partial: exact remediation required
Flag any TBD values, missing cross-references, or
incomplete sections with document and line reference.
```

→ **Phase 5 — Parallel Handoffs on ARB Approval:** Technical Lead (handoff brief + ADR rationale) · DevOps (resource specs + Infrastructure Guide) · Product Owner (Executive Summary + approval confirmation)

Key Takeaway

AI accelerates drafting — but architects ensure correctness.

AI helps you write faster. AI helps you structure your thinking.
But the **judgment** , the **decisions** , the **governance** — that's still ours.
And the right tool for the right document makes all the difference.


Copilot Squad
Collaborating Beyond Boundaries
Governance-Review
Boundaries

From One Agent to Many

Single-agent workflows have a ceiling. Squad breaks through it.

Single Agent

What We've Done Today

- **One context window** holds everything
- **Sequential:** requirements → options → review → docs
- **Context bleed:** earlier decisions pollute later reasoning
- **One model's perspective** on every task
- **Scales linearly** — more work, more time

VS

Copilot Squad

What's Next

- **Each agent has its own bounded context window**
- **Parallel:** specialized agents work simultaneously
- **No context bleed:** each agent sees only what it needs
- **Multiple model perspectives,** each tuned to its role
- **Scales with agent count** — more agents, not more time

*Squad isn't **faster AI.** It's a **different way of working.***

The Agent Roster

Five agents. Five domains. Each one bounded, focused, and auditable.

Agent	Role	Inputs	Outputs
Requirements Agent	Analyzes requirements, surfaces ambiguities, generates clarifying questions	Raw user stories, acceptance criteria	Structured analysis, clarifying questions, implicit NFRs
Design Agent	Generates architecture options with tradeoffs	Clarified requirements, constraints	Four architecture approaches with strengths, weaknesses, risks
Tradeoffs Agent	Evaluates options against quality attributes and team context	Design Agent output, team profile	Ranked options with quality attribute scores, recommendation
Adversarial Agent	Stress-tests the preferred architecture	Tradeoffs Agent recommendation	Severity-rated findings, failure scenarios, remediation suggestions
Documentation Agent	Generates APS-compliant documentation from template	All prior agent outputs, template files	Architecture Guide draft, ADR drafts, Risk Register seeds

Together: **Requirements → Design → Tradeoffs → Adversarial Review → Documentation.** *In parallel.*

The Handoff Model — Context Without Bleed

What each agent knows — and what it doesn't.

Requirements Agent → *structured analysis*

HANDOFF: structured analysis only

Design Agent → *four architecture approaches*

HANDOFF: requirements analysis + architecture approaches

Tradeoffs Agent → *ranked options with recommendation*

HANDOFF: requirements + recommended approach

Adversarial Agent → *severity-rated findings*

HANDOFF: all structured outputs

Documentation Agent → *APS document drafts*

Key Principles

- **Structured outputs only** — each agent receives structured outputs, not conversation transcripts
- **Bounded scope** — prevents context bleed between agent roles
- **Human-readable handoffs** — each handoff payload is auditable and reviewable
- **Architect reviews at every handoff** — not just at the end of the pipeline

Demo Setup: Copilot Squad

Preview

GitHub Copilot Squad

Input: *"As a system user, I want to receive timely notifications through my preferred channel when key events occur, so that I can stay informed and respond promptly."*

What We'll See

1. **Squad initialization** — agent roster configured with scoped instructions
2. **Requirements Agent** running against the Notification Service story
3. **Design Agent** receiving the handoff and generating architecture options
4. **Documentation Agent** receiving combined output and generating Phase 1 draft documents
5. **Human review gates** between each agent

What to Watch

- **Each agent's context is bounded** — no context bleed between roles
- **Handoff payload is structured JSON** — not a conversation transcript
- **Documentation Agent uses APS template files directly** — grounded in your governance structure
- **Each agent output is reviewable** before the next agent runs

> ⚠ **Note:** If live Squad is unavailable due to preview constraints, an annotated walkthrough of a pre-run session will be shown.

Live Demo

Preview

9

Copilot Squad

Notification Service — End-to-End

GitHub Copilot Squad

Switching to GitHub Copilot Squad...

Squad + Governance — Where Squad Feeds the APS

Squad Session
Requirements · Options · Adversarial · Phase 1 Drafts

→

Architect Review Gates
Reviewed · Corrected · Approved foundation

→

Jasper Architecture Process
Phase 2 onward — ADRs · Journeys · Infrastructure

→

ARB Gate
Final human governance checkpoint

⚠ Honest Limitations of Squad (Today)

- **Hallucination still occurs in each agent** — review gates are non-negotiable
- **Agents don't know your organization** — Work IQ is not yet integrated into Squad
- **Complex cross-agent reasoning is still immature** — agents can't revise each other's outputs
- **Preview capability** — not all features available to all users today

✓ What Squad Changes

- Generation time for **Phases 0–1 output drops from hours to minutes**
- The architect's role shifts from **drafter to reviewer** — a higher-leverage position
- The **quality floor rises** — every agent applies its full prompt consistently

Squad doesn't replace the APS. Squad accelerates the front end of it.

Bringing It All Together

Integrating Vision, Design, and Intelligence



The New Role of the Architect

01

AI Expands Possibilities

More options, more patterns, more tradeoffs, more risks explored — faster than ever before.

02

Humans Provide Judgment

We decide what is real, what is feasible, what is responsible — and we own the consequences.

03

The Partnership Matters

Human judgment plus AI acceleration — not replacement, not automation. Augmentation.

Not replacement. Not automation. Augmentation.

Closing Statement

You now have superpowers.

Use them to build better systems, better teams, and better futures.

Because AI doesn't replace architects — AI amplifies architects.

Cloud & AI Summit · September 30, 2026

Thank You

Q & A

Chad Green

Senior Systems Architect — Jasper Engines & Transmissions

linkedin.com/in/chadwickgreen github.com/talelearncode chadgreen@chadgreen.com

Find me after the session — I will talk about Copilot Squad all day if you let me.